

归并和计数排序

归并排序

考察归并排序的题目可以形态各异，但总会有万变不离其宗，希望看完今日之章，你能掌握归并排序及其思想大成。

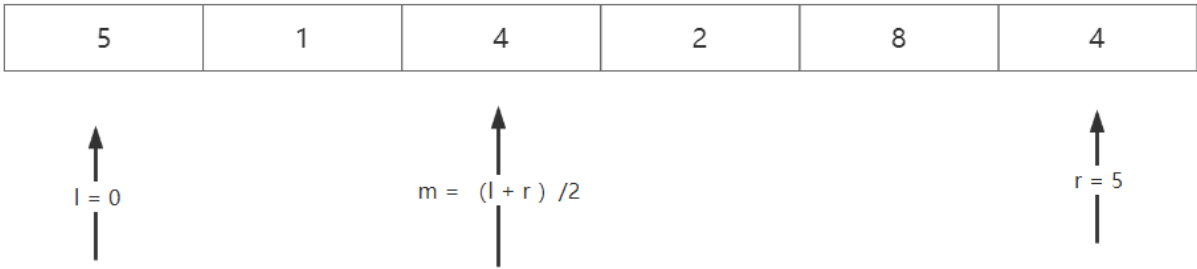
算法思想

归并排序和之前的冒泡排序、插入排序和选择排序不同，其蕴含了一种分治的思想。

话不都说，我们还是以数组 `arr = [5,1,4,2,8,4]` 为例进行一趟归并排序。

5	1	4	2	8	4
---	---	---	---	---	---

第一步：计算数组的中间元素 $m = (l + r) / 2 = 2$ ，然后将数组分成 $[l, m]$ 和 $[m + 1, r]$ 两个区间，即 $[0, 2]$ 与 $[3, 5]$ 两个子数组，这一步属于「分」的过程。



第二步：计算分出来的子数组 $[0, 2]$ 的中间元素， $m = (l + r) / 2 = 1$ ，将数组分成了 $[0, 1]$ 以及 $[2]$ 两个子数组，这一同样是「分」的过程，但同时也应该注意到，这一步和第一步的操作过程是一致的，也就说第一步和第二步是同一个功能，所以最终你会在代码中看到递归实现，后面会专门讲归并排序所蕴含的递归思想。

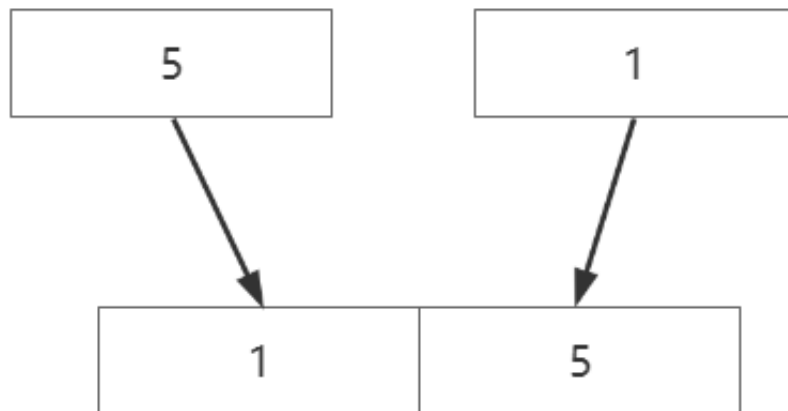
5	1	4
---	---	---

5	1	4
---	---	---

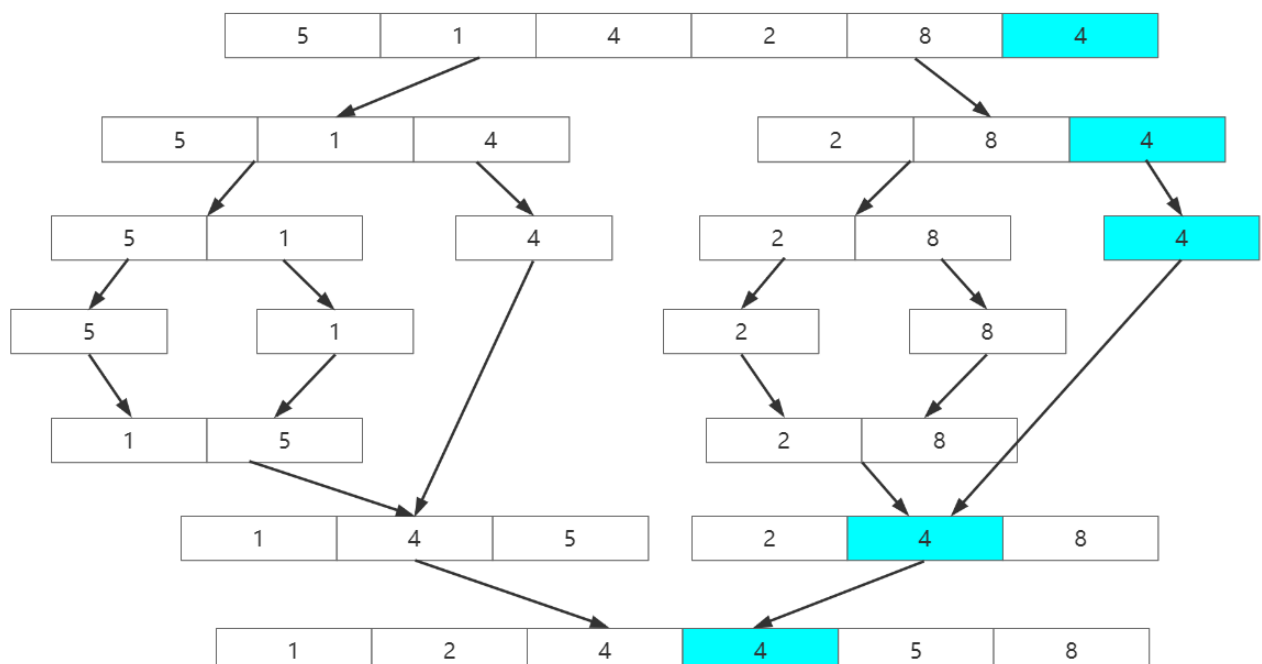
第三步：将上一步分出的子数组 $[0, 1]$ 再进行拆分， $m = 0$ ，将数组分成了 $[0]$ 和 $[1]$ ，只包含一个元素。



第四步：将 `arr[0]` 和 `arr[1]` 进行合并，因为上一步得到的两个数组不可再分了，已经有序，所以进行合并。这一步事实上也是「治」的过程，所谓治就是真正进行排序的过程，因为通过这一步元素 5 和 1 的顺序将被改变。



之后的每一步都是如此，我们在下图中将每一步用红色数字标注了出来：



第二步：遍历数组 `[1,4,1,2,5,2,4,1,8]`，访问第一个元素 `1`，然后将数组 `count[]` 中下标为 `1` 的元素加 1，表示当前 `1` 出现了一次，即 `count[1] = 1`；

1	4	1	2	5	2	4	1	8
0	1	0	0	0	0	0	0	0

第三步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第二个元素 `4`，然后将数组 `count[]` 中下标为 `4` 的元素加 1，表示当前访问的元素 `4` 当前出现了 1 次，即 `count[4] = 1`；

1	4	1	2	5	2	4	1	8
0	1	0	0	1	0	0	0	0

第四步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第三个元素 `1`，然后将数组 `count[]` 中下标为 `1` 的元素加 1，即 `count[1] = 2`，表示当前 `1` 出现了 2 次：

1	4	1	2	5	2	4	1	8
0	2	0	0	1	0	0	0	0

第五步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第四个元素 `2`，然后将数组 `count[]` 中下标为 `2` 的元素加 1，即 `count[2] = 1`，表示当前 `2` 出现了 1 次：

1	4	1	2	5	2	4	1	8
0	2	1	0	1	0	0	0	0

第六步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第五个元素 `5`，然后将数组 `count[]` 中下标为 `5` 的元素加 1，即 `count[5] = 1`，表示当前 `5` 出现了 1 次：

1	4	1	2	5	2	4	1	8
0	2	1	0	1	1	0	0	0

第七步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第六个元素 `2`，然后将数组 `count[]` 中下标为 `2` 的元素加 1，即 `count[2] = 2`，表示当前 `2` 出现了 2 次：

1	4	1	2	5	2	4	1	8
0	2	2	0	1	1	0	0	0

第八步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第七个元素 **4**，然后将数组 `count[]` 中下标为 **4** 的元素加 1，即 `count[4] = 2`，表示当前 **4** 出现了 **2** 次：

1	4	1	2	5	2	4	1	8
0	2	2	0	2	1	0	0	0

第九步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第八个元素 **1**，然后将数组 `count[]` 中下标为 **1** 的元素加 1，即 `count[1] = 3`，表示当前 **1** 出现了 **3** 次：

1	4	1	2	5	2	4	1	8
0	3	2	0	2	1	0	0	0

第十步：访问数组 `[1,4,1,2,5,2,4,1,8]` 的第九个元素 **8**，然后将数组 `count[]` 中下标为 **8** 的元素加 1，即 `count[8] = 1`，表示当前 **8** 出现了 **1** 次：

1	4	1	2	5	2	4	1	8
0	3	2	0	2	1	0	0	1

此时遍历数组 `[1,4,1,2,5,2,4,1,8]` 结束，我们得到了一个 `count[]` 数组，而只要得到了这个 `count[]` 数组，我们的排序算法就相当于结束了，接下来的就只是输出了。

稳定性

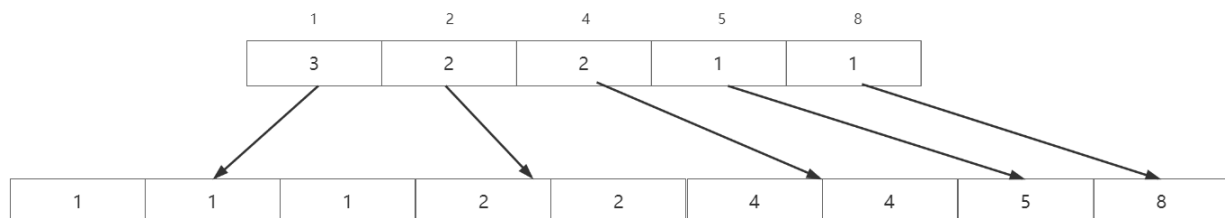
如果不考虑计数排序的稳定性，我们按照数组 `count[]` 中对应下标的出现次数直接输出即可：

为了保证计数排序的稳定性，我们又该如何做呢？

先不考虑这么复杂，但是从宏观的角度来看，我们的目的就是找到待排序数组中每一个元素在排序后数组当中的正确位置。首先看一下 `count[]` 数组本身，数组中的 **0** 对于我们的输出没有任何影响，所以我们可以考虑将其直接去掉：

0	3	2	0	2	1	0	0	1
1	2	4	5	8				
3	2	2	1	1				

那么此时的我们就可根据去掉之后的数组得到排序后数组的一个轮廓图：



但是这样我们并不知道相同的数字在对应原始数组 `arr[]` 中的哪一个元素，就相当于直接输出，而没有考虑元素的相对顺序；但是对这个过程的理解有助于我们接下来理解稳定性的处理过程。

我们看到，数组 `count[]` 中的每一个值表示它所对应的下标在排序后数组的出现次数，那么我们遍历数组（下标从 1 开始），并对数组 `count[]` 中的每一个元素执行 `count[i] = count[i] + count[i-1]` 会得到什么呢？

0	3	2	0	2	1	0	0	1
0	3	5	5	7	8	8	8	9

此时得到新的 `count[]` 将表示他们的位置信息，比如 3 表示它的下标 1 一定出现在前 3 的位置；而紧接其后 5 则表示下标 2 出现在第 4 和第 5 个位置；下标为 3 的 `count[3] = 5`，其与前面的 `count[2]` 值相同，两者之差也就表示其出现次数，0 次，所以不占位置；下标为 4 的位置值为 7，则表示下标 4 出现在第 6 和 7 的位置，依次类推，你也可以对新的 `count[]` 数组中的每一个元素做出解释。

但我们怎么可能停留在这里呢？

有了这个新的 `count[]` 数组，我们如何得到元素数组 `arr[]` 在排序后的输出数组 `output[]` 中的正确位置呢？

回答了这个问题，稳定的计数排序也就彻底理解了~~

第一步：从后向前遍历，具体为什么是从后向前，看完了你就会明白了！首先是 `i = n-1 = 8`，然后计算出 `arr[i] = arr[8] = 8` 在排序后数组的正确位置 `count[arr[i]] - 1 = count[8] - 1 = 8`，即排序后 `arr[8]` 的正确位置为 8，然后将 `arr[8]` 赋值给 `output[8] = 8`，但是 `count[arr[8]] = count[8]` 减 1：

1	4	1	2	5	2	4	1	8
0	3	2	0	2	1	0	0	1
0	3	5	5	7	8	8	8	9
								8

`count[arr[i]] - 1`

第二步：`i = n - 2 = 7`，然后计算 `arr[7] = 1` 在排序后数组的正确位置 `count[arr[7]] - 1 = count[1] - 1 = 2`，即最后一个 1 在排序后的正确位置下标为 2，然后将 `count[arr[7]]` 的值减 1。这里为什么要减 1，因为我们已经找到了最后一个 1 的正确位置，目前就剩余两个 1 没有找到正确位置。

1	4	1	2	5	2	4	1	8
0	3	2	0	2	1	0	0	1
0	3	5	5	7	8	8	8	9
							2	8

count[arr[i]] - 1

依次类推，我们就可以得到arr[]中每一个元素在排序后的正确位置。

1	4	1	2	5	2	4	1	8
0	3	2	0	2	1	0	0	1
0	3	5	5	7	8	8	8	9
0	5	1	3	7	4	6	2	8

count[arr[i]] - 1

1	1	1	2	2	4	4	5	8
---	---	---	---	---	---	---	---	---

这就是稳定的计数排序，那我们来再回答一下为什么从后向前遍历新的 count[] 数组？

因为只有这样才能保证计数排序的稳定性！比如原始数组 arr[] 中 3 个 1 的在排序后的相对位置就没有发生变化，依旧保持：

可是问题又来了，如果我们的数组变成了 arr[] = {-1,4,1,-2,5,-2,4,-1,8}，上面介绍的计数排序的实现方式不就出现了问题吗？因为数组下标也没有负数的情况呀！很简单，将最小元素映射到下标为0的位置。其他元素依次映射。

我们只需要找到数组 arr[] = {-1,4,1,-2,5,-2,4,-1,8} 中的最小值 min = -2，以及最大值 max = 8，然后开辟一个大小为 max - min + 1 的 count[] 数组，统计出数组当中每一个元素出现的次数即可，就像下面这样：

-1	4	-1	-2	5	-2	4	-1	8	
2	3	0	0	0	2	1	0	0	1

count[arr[i] - min]

其中数组 arr[] 的最小值 min = -2，-2 被映射到了 count[] 数组下标为 0 的位置，原数组中包含 2 个 -2，所以 count[0] = 2；原数组 arr[] 当中有 3 个 -1，其中 -1 - (-2) = 1，也就是说 -1 映射到了 count[] 数组下标为 1 的位置，所以 count[1] = 3。

复杂度分析

时间复杂度

在整个代码实现过程中，我们仅仅出现了一层的 for 循环，没有出现任何 for 循环的嵌套，所以计数排序的时间复杂度为 $O(n)$ 量级。

空间复杂度

由于计数排序过程中，我们使用到了一个 `max - min + 1` 大小的 `count[]` 数组，所以计数排序的空间复杂度为 $O(n)$ 量级。

优缺点分析

1. 如果输入数据的范围 `range = max - min + 1` 不明显大于要待排序数组的长度 `n = arr.length`，则计数排序是相当高效的，比时间复杂度为 $O(n^2)$ 的快速和归并排序都优秀。
2. 计数排序不是基于比较的排序算法，时间复杂度为 $O(n)$ ，空间复杂度与数据范围成正比。
3. 计数排序通常用作另一个排序算法（例如基数排序）的子过程。
4. 计数排序可以使用部分哈希（partial Hashing）在 $O(1)$ 的时间内统计数据的频率。
5. 计数排序适用于负输入。
6. 计数排序不适用于小数的情况。

桶排序

在数据结构与算法的排序中，桶排序是所有排序中最简单的排序之一。并且桶排序和 计数排序，基数排序 有很多相似和渊源之处。

桶排序的思想

其实桶排序重要的是它的思想，而不是具体实现，桶排序从字面的意思上看：

- 1、若干个桶，说明此类排序将数据放入若干个桶中。
- 2、每个桶有容量，桶是有一定容积的容器，所以每个桶中可能有多个元素。
- 3、从整体来看，整个排序更希望桶能够更匀称，即既不溢出(太多)又不太少。

那么这些个桶跟排序有什么关系呢？我们先来看桶排序的定义

工作的原理是将数组分到有限数量的桶子里。每个桶子再个别排序（有可能再使用别的排序算法或是以递归方式继续使用桶排序进行排序）。桶排序是鸽巢排序的一种归纳结果。当要被排序的数组内的数值是均匀分配的时候，桶排序使用线性时间 $O(n)$ 。但桶排序并不是 比较排序，他不受到 $O(n \log n)$ 下限的影响。

简单点说：

- 1、将待排序的序列部分分到若干个桶中，每个桶内的元素再进行个别的排序。
- 2、时间复杂度最好可能是线性的 $O(n)$ ，桶排序不是基于比较的排序

当然，桶排序是一种用空间换取时间的排序。

既然是排序，那么最终的结果肯定是从小到大的，桶排序借助桶的位置完成一次初步的排序——将待排序元素分别放至各个桶内。

而我们通常根据待排序元素整除的方法将其较为均匀的放至桶中，如 8 5 22 15 28 9 45 42 39 19 27 47 12 这个待排序序列，假设放入桶编号的规则为： $n/10$ 。这样首先各个元素就可以直接通过整除的方法放至对应桶中。而右侧所有桶内数据都比左侧的要大！

8	5	22	15	28	9	45	42	39	19	27	12
---	---	----	----	----	---	----	----	----	----	----	----

8 5 9	15 19 12	22 28 27	39	45 42 47
-------	----------	----------	----	----------

在刚刚放入桶中的时候，各个桶的大小相对可以确定，右侧都比左侧大，但桶内还是无序的，对各个桶内分别进行排序，再依次按照桶的顺序、桶内序列顺序得到一个最终排序的序列。

关于桶排序的算法分析

上面讲了桶排序的具体思想，但是你是不是总觉得心理没那么踏实呢，这就完了？总觉得怪怪的是吧？

桶排序确实有很多不一样的地方，无论是算法时间复杂度还是整个算法的流程，都不如啥快排啦、归并啦这些传统排序来的实在。

桶排序的时间复杂度到底是多少？

我们假设有 n 个待排序数字。分到 m 个桶中，如果分配均匀这样平均每个桶有 n/m 个元素。首先在这里我郑重说明一下桶排序的算法时间复杂度有两部分组成：

- 1.遍历处理每个元素， $O(n)$ 级别的普通遍历
- 2.每个桶内再次排序的时间复杂度总和

对于第一个部分，我想大家都应该理解最后排好序的取值遍历一趟的 $O(n)$ ；而第二部分咱们可以进行这样的分析：

如果桶内元素分配较为均匀假设每个桶内部使用的排序算法为快速排序，那么每个桶内的时间复杂度为 $(n/m) \log(n/m)$ 。有 m 个桶，那么时间复杂度为 $m * (n/m) \log(n/m) = n (\log n - \log m)$ 。

在这里如果到达极限情况 $n=m$ 时。就能确保避免桶内排序，将数值放到桶中不需要再排序达到 $O(n)$ 的排序效果，当然这种情况属于计数排序，后面再详解计数排序记得再回顾。

桶排序适用情况

桶排序并且像常规排序那样没有限制，桶排序有相当的限制。因为桶的个数和大小都是我们人为设置的。而每个桶又要避免空桶的情况。所以我们在使用桶排序的时候即需要对待排序数列要求偏均匀，又要要求桶的设计兼顾效率和空间。

桶排序的适用场景非常明了，那就是在数据分布相对比较均匀或者数据跨度范围并不是很大时，排序的速度还是相当快且简单的。

但是当数据跨度过大时，这个空间消耗就会很大；如果数值的范围特别大，那么对空间消耗代价肯定也是不切实际的，所以这个算法还有一定的局限性。同样，由于时间复杂度为 $O(n+m)$ ，如果 m 比 n 大太多，则从时间上来说，性能也并不是很好。

但是实际上在使用桶排序的过程中，我们会使用类似散列表的方式去实现，这时的空间利用率会高很多，同时时间复杂度会有一定的提升，但是效率还不错。

我们在开发过程中，除了对一些要求特别高并且数据分布较为均匀的情况使用桶排序，还是很少使用桶排序的，所以即使桶排序很简单、很快，我们也很少使用它。

桶排序更多地被用于一些特定的环境，比如数据范围较为局限或者有一些特定的要求，比如需要通过哈希映射快速获取某些值、需要统计每个数的数量。但是这一切都需要确认数据的范围，如果范围太大，就需要巧妙地解决这个问题或者使用其他算法了。

基数排序

与基于比较的排序算法（归并排序、堆排序、快速排序、冒泡排序、插入排序等等）相比，基于比较的排序算法的时间复杂度最好也就是 $O(n \log n)$ ，而且不能比 $O(n \log n)$ 更小了。

计数排序（Counting Sort）的时间复杂度为 $O(n)$ 量级，更准确的说，计数排序的时间复杂度为 $O(n+k)$ ，其中 k 表示待排序元素的取值范围（最大与最小元素之差加 1）。

那么问题来了，当这个元素的范围在 1 到 n^2 怎么办呢？

此时就不能用计数排序了奥，因为这种情况下，计数排序的时间复杂度达到了 $O(n^2)$ 量级。

比如对数组 `[170, 45, 75, 90, 802, 24, 2, 66]` 这个而言，数组总共包含 8 个元素，而数组中的最大值和最小值之差为 `802 - 2 = 800`，这种情况下，计数排序就“失灵了”。

那么有没有那种排序算法可以在线性时间对这个数组进行排序呢？

答案就是今天要讲的 **基数排序（Radix Sorting）**。基数排序的总体思想就是从待排序数组当中，元素的最低有效位到最高有效位 **逐位** 进行比较排序；此外，基数排序使用计数排序作为一个排序的子过程。

下面我们就以数组 `[170, 45, 75, 90, 802, 24, 2, 66]`，快乐学习基数排序！

算法思想

数组当中的最大值 **802** 为三位数，所以我们在不足三位的数字前面补 **0**，即 `[170, 045, 075, 090, 802, 024, 002, 066]` 方便后续说明基数排序。

170	45	75	90	802	24	2	66
170	045	075	090	802	024	002	066

第一步：按数组当中元素的最低有效位，个位 `[170, 045, 075, 090, 802, 024, 002, 066]`，进行计数排序：

090	002	075							
170	802	024	045	066					

第二步：按数组当中元素的次最低有效位，十位数，进行计数排序：

002						075			
802	024	045	066	170	090				

802 002 024 045 066 170 075 090

第三步：按数组当中元素的最高有效位，百位，进行计数排序：

090									
075									
066									
045									
024									
002	170							802	

002 024 045 066 075 090 170 802

这样我们就完成了基数排序，得到了一个有序数组 **[2, 24, 45, 66, 75, 90, 170, 802]** 。

复杂度分析

时间复杂度

设d表示输入的数组当中最大值的位数（比如 802，3位， $d = 3$ ），那么基数排序的时间复杂度就是 $O(d \times (n+b))$ ，其中n表示数组的长度，而b则是表示一个数的基础，对十进制而言 $b = 10$ ；

设K是一个计算机可表示的最大整数，那么 $d = \log_{\text{以}b\text{为底}} K$ ；

则基数排序整个时间复杂度为 $O((n+b) \times \log_{\text{以}b\text{为底}} K)$ 。

对于一个较大的数 K，计数排序的这个时间复杂度似乎比基于比较的排序算法都慢，但是事实未必，接着看。

假设 $K \leq n^c$ 取一个最大整数，其中c是一个常量；

那么基数排序的时间复杂度就为 $O(c \times (n+b) \times \log_{\text{以}b\text{为底}} K)$ ，其中b 和 c都是常数，我们可以忽略不计，那么基数排序的时间复杂度就变成了 $O(n \times \log_{\text{以}b\text{为底}} n)$ ，但是这依旧不比基于排序算法的最好时间复杂度 $O(n \log n)$ 好呀？

可是当我们将这个b取足够大呢？b取多少的时候基数排序的时间复杂度才能变成线性呢？

当 $b = n$ 的时候， $\log n$ 为底 $n = 1$ ，基数排序的时间复杂度就变成了 $O(n)$ ，线性时间复杂度。

也就是说，当数字用 n 进制表示的时候，我们就可以对 1 到 n^c 范围内的数组进行线性排序。

对于元素的跨度（范围）比较大的数组而言，基数排序的运行时间可能比快速排序要好。但是对于基数排序而言，其渐近时间复杂度中隐含了更高的常量因子，并非完全的线性；而快速排序更有效地利用硬件缓存，提高其效率。此外，基数排序使用计数排序作为子过程，计数排序占用额外的空间来对数组进行排序。

但是基数排序解决我们最开始所提出的问题，当数据范围在 1 到 n^2 时，计数排序的复杂度将变为 $O(n^2)$ 量级，而基数排序依旧可以在线性时间进行排序！

空间复杂度

计数排序使用了额外的空间进行排序，空间复杂度为 $O(n)$ 。